XYZ University / Institute of Technology

Department of Computer Engineering

Academic Year 2023–2024

BACHELOR OF ENGINEERING

FINAL YEAR PROJECT REPORT

CBT Software

Computer Based Testing System

A web-based examination management platform with AI-powered question generation

Submitted By	:	Himanshu Singh & Akash Anand
Roll Number	:	[Your Roll Numbers]
Branch	:	Computer Engineering
Guide	:	[Guide Name, Designation]
Submitted To	:	[University / Examination Board]
Date	:	February 2024

CERTIFICATE

Department of Computer Engineering

[Institute Name], [City] — [PIN]

This is to certify that the project report entitled **"CBT Software — Computer Based Testing System"** submitted by **Himanshu Singh** (Roll No. _____) and **Akash Anand** (Roll No. _____) of Final Year B.E. (Computer Engineering) is a bonafide record of the work carried out by them under our guidance and supervision.

The project has been carried out in partial fulfillment of the requirements for the award of the degree of **Bachelor of Engineering in Computer Engineering** from [University Name] for the academic year 2023–2024.

To the best of our knowledge and belief, the work embodied in this report has not been submitted to any other university or institution for the award of any degree or diploma.

Project Guide	**Head of Department**	**Principal**
[Name]	[Name]	[Name]
[Designation]	Dept. of Computer Engineering	[Institute Name]

Himanshu Singh	**Akash Anand**

Place: [City] **Date:** ____ / ____ / 2024

Acknowledgement

We wish to express our deep sense of gratitude and sincere thanks to our project guide **[Guide Name]**, [Designation], Department of Computer Engineering, [Institute Name], for their invaluable guidance, constant encouragement, and constructive suggestions throughout the course of this project work.

We are grateful to **[HoD Name]**, Head of Department of Computer Engineering, for providing us with all the necessary facilities and support for the completion of this project.

We are also thankful to all the faculty members of the Department of Computer Engineering for their direct and indirect support and encouragement.

We extend our heartfelt thanks to our family and friends for their moral support and motivation throughout our project work.

Himanshu Singh

Akash Anand

Abstract

CBT Software — Computer Based Testing System

The CBT (Computer Based Testing) Software is a full-stack web application designed to digitize and streamline the entire lifecycle of computer-based examinations. The system provides a multi-role platform supporting four user types: Super Admin, Company (exam organizer), Exam Center, and Student.

Built using the Python Django framework with an SQLite3 database and Bootstrap 5 frontend, the system enables exam companies to create and manage tests, build a question bank (including AI-powered automatic question generation via the OpenAI API), and register multiple exam centers with secure hashed credentials.

Exam centers log in to manage student entry, seating arrangements, and emergency procedures. Students take fully timed Multiple Choice Question (MCQ) exams through an AJAX-driven interface that supports question navigation, review marking, and auto-submission on timer expiry.

The system implements session-based authentication, PBKDF2-SHA256 password hashing, Django REST Framework for API endpoints, and an optional OpenAI integration for automated question generation categorized by topic and difficulty level.

Keywords: Computer Based Testing, Django, MCQ Examination, AI Question Generation, Exam Management System, Online Examination, Web Application.

Table of Contents

1. Introduction

1.1 Problem Statement 01

1.2 Objectives 02

1.3 Scope of the Project 02

1.4 Organization of Report 03

2. Literature Survey

2.1 Existing Systems 04

2.2 Comparison with Proposed System 05

3. System Design

3.1 System Architecture 06

3.2 User Roles and Access Control 07

3.3 Data Flow Diagram 08

3.4 Entity Relationship Diagram 09

3.5 Database Design 10

3.6 Module Description 11

4. Implementation

4.1 Technologies Used 13

4.2 Development Environment 14

4.3 Module Implementation 14

4.4 AI Question Generation 17

4.5 Security Implementation 18

5. Testing

5.1 Testing Methodology 19

5.2 Test Cases 20

6. Conclusion and Future Work

6.1 Conclusion 22

6.2 Future Enhancements 22

References **23**

Chapter 1: Introduction

1.1 Problem Statement

Traditional paper-based examinations are associated with significant challenges: high printing and logistics costs, susceptibility to paper leaks, manual and error-prone evaluation, slow result declaration, limited scalability, and large administrative overhead. These problems are especially acute for organizations that conduct examinations at multiple geographically distributed centers simultaneously.

There is a clear need for a reliable, scalable, and secure digital platform that automates question bank management, examination delivery, multi-center coordination, and result processing — while remaining easy to deploy and maintain by non-technical administrators.

1.2 Objectives

- Design and develop a multi-role web-based examination management system.
- Provide a secure company portal for creating tests, managing question banks, and registering exam centers.
- Provide a center portal for managing student entry and seat arrangements.
- Deliver an AJAX-driven student exam interface with countdown timer, question palette, and review marking.
- Implement secure password storage using PBKDF2-SHA256 hashing.
- Integrate an AI-powered question generation module using the OpenAI API.
- Build RESTful API endpoints using Django REST Framework.
- Ensure role-based access control through custom session management.

1.3 Scope of the Project

The CBT Software covers the following scope:

- **In scope:** MCQ-based examinations, multi-center management, question bank with AI generation, timed exam interface, session-based security, Django admin panel.
- **Out of scope:** Video proctoring, descriptive answer evaluation, SMS/email notifications, payment gateway (these are planned as future enhancements).

1.4 Organization of Report

Chapter 2 surveys existing examination systems and motivates the proposed solution. **Chapter 3** covers system design including architecture, DFD, ER diagram, and module descriptions. **Chapter 4** describes the technology stack and implementation details. **Chapter 5** presents testing methodology and test cases. **Chapter 6** concludes the report and outlines future work.

Chapter 2: Literature Survey

2.1 Existing Systems

2.1.1 Google Forms / Microsoft Forms

Cloud-based quiz tools that allow basic MCQ creation and automatic grading. However, they lack multi-center management, exam-center-level security, a dedicated student interface with question palettes, or integration with an organizational exam workflow. They are not suitable for formal, proctored examinations at scale.

2.1.2 Moodle (LMS)

An open-source Learning Management System supporting online quizzes. While feature-rich, Moodle requires complex installation, has a steep learning curve for administrators, and is primarily designed for course management rather than standalone examination delivery.

2.1.3 HackerRank / TestGorilla

Commercial SaaS platforms for technical assessments. They are proprietary, subscription-based, and not customizable for local institutional requirements such as custom center management or government examination workflows.

2.1.4 Conducted Research

Several papers (Nwaocha, V.O. — 2012; Jimoh, R.G. et al. — 2012) highlight that computer-based testing reduces examination malpractice, speeds up result processing, and improves test validity compared to paper-based methods. AI-assisted question generation has been explored in recent works (Kurdi et al., 2020), demonstrating that NLP-based question generation significantly reduces content creation time for educators.

2.2 Comparison with Proposed System

Feature	Google Forms	Moodle	CBT Software
Multi-center management	✗	Partial	✓
Custom exam company portal	✗	✗	✓

Feature	Google Forms	Moodle	CBT Software
Dedicated student exam UI	Partial	✓	✓
Question palette & review	✗	✓	✓
AI question generation	✗	✗	✓
PBKDF2 password hashing	N/A	✓	✓
REST API	✗	Partial	✓
Open source & self-hosted	✗	✓	✓
Lightweight setup	✓	✗	✓

Chapter 3: System Design

3.1 System Architecture

The CBT Software follows a monolithic Django architecture with five Django applications (apps), each encapsulating a distinct business domain. The entire application is served by the Django development server (or Gunicorn/Nginx in production) and uses SQLite3 as the relational database.

```
┌──────────────────────────────────────────────────────────────────┐
│                      Client Browser (HTTP)                       │
└──────────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────────┐
│                     Django Application Server                    │
│  ┌──────────────┐  ┌────────────────┐  ┌──────────────────────┐ │
│  │general_zone│  │ companylogin │  │     centerlogin      │ │
│  │(Auth/Public)│  │(Company Dash)│  │(Center Management)   │ │
│  └──────────────┘  └────────────────┘  └──────────────────────┘ │
│  ┌──────────────────────────┐  ┌──────────────────────────────┐  │
│  │      studentexam         │  │        admin_zone            │  │
│  │   (Exam Interface)       │  │   (Super Admin / REST API)   │  │
│  └──────────────────────────┘  └──────────────────────────────┘  │
└──────────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────────┐
│                   SQLite3 Database (db.sqlite3)                  │
│  Tables: Test | Center | Question | Company | Admin | ...        │
└──────────────────────────────────────────────────────────────────┘
```

Figure 3.1 — System Architecture Diagram

3.2 User Roles and Access Control

Role	Authentication	Session Key	Capabilities
Super Admin	Django built-in	Django session	Full access via /admin/ — manage all models
Company	Email + hashed password	`com_id`	Create/delete tests, manage centers and question bank

Role	Authentication	Session Key	Capabilities
Center	Email + hashed password	`center_id`	Entry login, student list, seat arrangement, emergency
Student	Center-provided credentials	N/A (stateless)	Take timed MCQ exam, submit answers

Access control is enforced in each view by checking the relevant session key (e.g., `request.session['com_id']`) and redirecting unauthenticated requests to the homepage.

3.3 Data Flow Diagram (Level 0 — Context Diagram)

```
                    ┌──────────────┐
                    │   COMPANY    │
                    └──────┬───────┘
          [Login/Test/     │          [Dashboard/Test List/
           Questions/      │           Centers/Questions]
           Centers]        │
                           ▼
                    ┌──────────────────────┐
                    │                      │
┌──────────┐        │                      │        ┌──────────┐
│ CENTER   │──────► │     CBT SOFTWARE     │◄───────│  ADMIN   │
└──────────┘        │      SYSTEM          │        └──────────┘
[Login/Entry]       │                      │   [Manage All Data]
                    └──────────┬───────────┘
          [Student             │          [Exam Questions/
           Credentials/        │           Timer/Results]
           Instructions]       │
                               ▼
                    ┌──────────────┐
                    │   STUDENT    │
                    └──────────────┘
```

Figure 3.2 — Context Level DFD

3.4 Entity Relationship Diagram

```
┌──────────────┐          ┌──────────────┐          ┌──────────────────┐
│   COMPANY    │  1    *  │     TEST     │  1    *  │    QUESTION      │
│──────────────│──────────│──────────────│          │──────────────────│
│ id (PK)      │          │ id (PK)      │          │ id (PK)          │
│ name         │          │ test_name    │          │ question         │
│ email        │          │ no_of_questions│        │ option_1..4      │
│ phone        │          │ total_marks  │          │ correct_option   │
│ password     │          │ status       │          │ marks            │
└──────────────┘          └──────────────┘          │ topic            │
                                                    │ difficulty       │
┌──────────────┐                                    │ generated_by     │
│    CENTER    │                                    │ created_at       │
│──────────────│                                    └──────────────────┘
│ id (PK)      │
│ center_name  │
│ address      │
│ phone        │
│ email        │
│ profile_pic  │
```

```
│ password      │
└───────────────┘
```

Figure 3.3 — Entity Relationship Diagram

3.5 Database Design

Table: companylogin_test

Column	Type	Constraints	Description
id	INTEGER	PK, Auto-increment	Primary key
test_name	VARCHAR(128)	NOT NULL	Exam name
no_of_questions	INT	NOT NULL, > 0	Total questions
total_marks	INT	NOT NULL, > 0	Maximum marks
status	VARCHAR(128)	DEFAULT 'Active'	Active or Completed

Table: companylogin_center

Column	Type	Constraints	Description
id	INTEGER	PK, Auto-increment	Primary key
center_name	VARCHAR(256)	NOT NULL	Center name
address	VARCHAR(256)	NOT NULL	Physical address
phone	VARCHAR(256)	NOT NULL	Contact number
email	VARCHAR(256)	NOT NULL	Login email
profile_pic	VARCHAR(100)	NULLABLE	Uploaded image path
password	VARCHAR(128)	NOT NULL	PBKDF2-SHA256 hash

Table: companylogin_question

Column	Type	Description
id	INTEGER PK	Primary key
question	VARCHAR(512)	Question text

Column	Type	Description
option_1..4	VARCHAR(256)	Four MCQ options
correct_option	VARCHAR(256)	Correct answer text
marks	INT	Marks for correct answer
topic	VARCHAR(128) NULL	Subject topic
difficulty	VARCHAR(64) NULL	Easy / Medium / Hard
question_type	VARCHAR(64)	Default: MCQ
generated_by	VARCHAR(64) NULL	'manual' or 'ai'
created_at	DATETIME	Auto timestamp

3.6 Module Description

The system is divided into five Django application modules:

Module 1: general_zone (Public & Authentication)

- Serves the public-facing website (homepage, about, services, courses)
- Handles Company signup and login (with PBKDF2 password hashing)
- Handles Center login and Admin login
- Provides Contact/Request Demo form
- Exposes Company REST API via Django REST Framework

Module 2: companylogin (Company Portal)

- Company dashboard with quick-access cards
- Test management: create, list (ongoing/completed), delete
- Center management: add, list, delete
- Question bank: add manually, list with filters, delete, AI generation
- Session-secured views using `request.session['com_id']`

Module 3: centerlogin (Center Portal)

- Center dashboard
- Student entry login interface
- Student list management
- Seat arrangement visualization

- Emergency management page

Module 4: studentexam (Exam Interface)

- Student home with login form
- Instructions page with exam rules
- AJAX-powered running exam interface with countdown timer
- Question navigation (Next/Previous), Mark for Review, Clear Response
- Question palette showing answered/unanswered/reviewed status
- Submit confirmation page

Module 5: admin_zone (Super Admin)

- Super admin management panel
- Stores Company, Admin, Center list, and Contact data models
- Accessible via Django's built-in `/admin/` interface

Chapter 4: Implementation

4.1 Technologies Used

Component	Technology	Version	Purpose
Programming Language	Python	3.8+	Backend logic
Web Framework	Django	4.2.28	MVC framework, ORM, templating
REST API	Django REST Framework	3.15.2	API endpoints
Database	SQLite3	Built-in	Relational data storage
Frontend CSS	Bootstrap	5.x	Responsive UI components
Frontend JS	JavaScript (ES6)	–	AJAX exam navigation, modals
AI Integration	OpenAI API	≥ 1.0.0	Auto question generation
Password Security	Django Hashers	–	PBKDF2-SHA256 hashing
Async Server	ASGI (Django)	3.8.1	Async request support

4.2 Development Environment

Tool	Purpose
VS Code	Primary IDE
Python virtualenv	Isolated dependency environment
Git + GitHub	Version control
SQLite Browser	Database inspection

Tool	Purpose
Django Dev Server	Local development server
macOS / Windows	Development OS

4.3 Module Implementation

4.3.1 Authentication Flow

The system uses custom session-based authentication rather than Django's built-in authentication system. On successful login, the user's primary key is stored in the session:

```
request.session['com_id'] = user.id      # Company login
request.session['center_id'] = user.id   # Center login
```

Every protected view checks for this session key at the start:

```
if not request.session.get('com_id'):
    return redirect('homepage')
```

Passwords are hashed on registration using Django's `make_password()` which applies PBKDF2-HMAC-SHA256 with a random salt. Verification uses `check_password(raw, hash)`.

4.3.2 Test Management

A `Test` model stores the exam metadata. Tests are created via a POST form, listed on the ongoing/completed pages based on their `status` field, and deleted via a dedicated view that redirects based on the test's previous status.

4.3.3 AJAX Exam Interface

The running exam page loads questions dynamically via AJAX calls to two JSON endpoints: `/get-next-question/<id>/` and `/get-previous-question/<id>/`. The countdown timer is implemented in JavaScript using `setInterval`:

```
const timer = setInterval(() => {
  const now = new Date().getTime();
  const distance = endTime - now;
  if (distance <= 0) {
    clearInterval(timer);
    document.querySelector('form').submit();
  }
```

```
  // update display...
}, 1000);
```

4.4 AI Question Generation

When an `OPENAI_API_KEY` is configured, company users can auto-generate questions by submitting a topic, count, and difficulty level. The system sends a structured prompt to OpenAI's `gpt-3.5-turbo` model and parses the response:

```
prompt = f"Generate {count} MCQs on '{topic}' at {difficulty}
difficulty.
Provide each with options A, B, C, D and the correct letter."
```

The parsed questions are saved to the `Question` model with `generated_by='ai'` and are immediately available in the question bank. This feature is fully optional — the system functions normally without it.

4.5 Security Implementation

- **Password Hashing:** PBKDF2-SHA256 via Django's `make_password()`
- **CSRF Protection:** Django CSRF middleware enabled for all POST forms (`{% csrf_token %}`)
- **Session Security:** Custom session key checking on every protected view
- **No Raw Passwords Displayed:** Center passwords shown as `••••••••` in UI
- **Input Validation:** HTML5 `required` attributes and server-side Django ORM queries
- **SQL Injection Prevention:** All database access via Django ORM (parameterized queries)
- **XSS Prevention:** Django template auto-escaping enabled by default

Chapter 5: Testing

5.1 Testing Methodology

The project was tested using a combination of manual functional testing (black-box), integration testing across modules, and boundary value analysis. Django's built-in test framework was used for unit tests.

Testing Types Applied:

- **Unit Testing:** Individual view functions tested for correct HTTP responses
- **Integration Testing:** Login → Dashboard → Test Creation → Question Addition flow verified end-to-end
- **Boundary Testing:** Empty form submissions, invalid credentials, non-existent question IDs
- **Security Testing:** Session hijacking attempts, unauthorized URL access, CSRF token validation
- **UI Testing:** Responsive layout on desktop and mobile browsers

5.2 Test Cases

TC#	Module	Test Case	Input	Expected	Result
TC01	Company Login	Valid login	Correct email + password	Redirect to /com_dashboard/	PASS
TC02	Company Login	Invalid password	Wrong password	Redirect to homepage	PASS
TC03	Center Login	Valid center login	Correct email + password	Redirect to /entrylogin/	PASS
TC04	Add Test	Create new test	Test name, questions, marks	Test saved, redirect to dashboard	PASS
TC05	Delete Test	Delete active test	Test ID (active)	Test removed, redirect to ongoingtests	PASS

TC#	Module	Test Case	Input	Expected	Result
TC06	Add Center	Add new center	All required fields	Center saved with hashed password	PASS
TC07	Delete Center	Delete existing center	Center ID	Center removed, redirect to /centers/	PASS
TC08	Add Question	Manual question entry	All MCQ fields	Question saved with generated_by='manual'	PASS
TC09	Delete Question	Delete question	Question ID	Question removed, redirect to /questions/	PASS
TC10	Exam Interface	Next question AJAX	Current question ID	JSON response with next question	PASS
TC11	Exam Timer	Timer countdown	Page load	Timer decrements, auto-submit at 0	PASS
TC12	Security	Access dashboard without login	Direct URL /com_dashboard/	Redirect to homepage	PASS
TC13	Security	CSRF token missing	POST without CSRF token	403 Forbidden	PASS
TC14	Question Filter	Filter by topic	topic=Python	Only Python questions shown	PASS

Chapter 6: Conclusion and Future Work

6.1 Conclusion

The CBT Software successfully delivers a complete, self-contained Computer Based Testing platform that addresses the core challenges of paper-based examinations. The system implements the full examination lifecycle: from company registration and test/question creation, through multi-center management, to a secure timed MCQ examination interface for students.

Key accomplishments of this project include:

- A clean multi-role Django web application serving four distinct user types
- Secure authentication using PBKDF2-SHA256 password hashing and custom session management
- A responsive, modern dark-themed administrative dashboard
- An AJAX-driven timed exam interface with question palette, review marking, and auto-submission
- An AI-powered question generation module using the OpenAI API
- RESTful API endpoints using Django REST Framework
- Comprehensive CRUD operations for all core entities (Tests, Centers, Questions)

The project demonstrates the practical application of full-stack web development principles, database design, security best practices, and modern AI API integration within a real-world examination management context.

6.2 Future Enhancements

- **Online Proctoring:** Integration of webcam-based monitoring and AI-powered suspicious activity detection during exams.
- **Automated Result Calculation:** Immediate score computation after exam submission and a student results dashboard.
- **Multi-format Questions:** Support for True/False, fill-in-the-blank, and descriptive (essay) question types with manual grading.

- **Email Notifications:** Automated email alerts for exam schedules, center credentials, and result announcements.
- **Advanced Analytics:** Company-level dashboards showing question difficulty analysis, student performance graphs, and topic-wise breakdowns.
- **Mobile App:** React Native or Flutter mobile application for exam delivery on tablets.
- **Production Deployment:** Dockerized deployment with Gunicorn, Nginx, and PostgreSQL for production-grade reliability.
- **Question Negative Marking:** Configurable negative marking per question for competitive examination formats.

References

1. Nwaocha, V. O. (2012). Enhancing students' interest in mathematics via multimedia presentation. *African Journal of Mathematics and Computer Science Research*, 5(7), 129–136.
2. Jimoh, R. G., Shittu, A. J. K., & Kawu, Y. K. (2012). Students perception of computer based test (CBT) for examining undergraduate chemistry courses. *Journal of Emerging Trends in Educational Research and Policy Studies (JETERAPS)*, 3(2), 125–130.
3. Kurdi, G., Leo, J., Parsia, B., Sattler, U., & Al-Emari, S. (2020). A systematic review of automatic question generation for educational purposes. *International Journal of Artificial Intelligence in Education*, 30, 121–204.
4. Django Documentation. (2024). Django 4.2 Official Documentation. Retrieved from https://docs.djangoproject.com/en/4.2/
5. Django REST Framework. (2024). Django REST Framework Documentation. Retrieved from https://www.django-rest-framework.org/
6. OpenAI. (2024). OpenAI API Documentation. Retrieved from https://platform.openai.com/docs/
7. Bootstrap. (2024). Bootstrap 5 Documentation. Retrieved from https://getbootstrap.com/docs/5.0/
8. OWASP. (2023). OWASP Top Ten Web Application Security Risks. Retrieved from https://owasp.org/www-project-top-ten/
9. Holovaty, A., & Kaplan-Moss, J. (2009). *The Definitive Guide to Django: Web Development Done Right* (2nd ed.). Apress.
10. SQLite Documentation. (2024). SQLite Home Page. Retrieved from https://www.sqlite.org/docs.html

— End of Project Report —